MACHINE LEARNING WITH SCIKIT-LEARN

# Introduction to Machine Learning

01

What ML is, how it differs from classic programming, and your first model in five lines of scikit-learn.

**Instructor: Shutchon Premchaisawatt**

Master's Degree Program · Hands-on with Google Colab

AGENDA

# Today's agenda

01 **Why machine learning?**
Problems rules can't solve

02 **ML vs classic programming**
A different deal with the machine

03 **AI vs ML vs Deep Learning**
The map of the territory

04 **Three families of ML**
supervised · unsupervised · reinforcement

05 **The ML workflow**
From question to deployed model

06 **scikit-learn & the estimator API**
fit / predict / transform

07 **First model in 5 lines**
iris + KNeighborsClassifier

08 **Vibe coding for ML**
prompting AI assistants well

MOTIVATION

# Why machine learning?

Some problems are easy for humans, hopeless for hand-written rules.

### Spam filter

Write rules for every scam email? Scammers adapt overnight. Gmail learns from millions of labeled examples instead.

### Will this customer churn?

Hundreds of interacting signals — tenure, contract, complaints, charges. No human can write that rule book.

### Cat or dog?

You recognize them instantly, but can you write down how? This is the classic 'Polanyi's paradox'.

**Common thread: we can't specify the rules — but we have many examples of the answers.**

THE CORE IDEA

# Classic programming vs machine learning

## CLASSIC PROGRAMMING

**Rules**
written by you
→
**+ Data**
→
**= Answers**

## MACHINE LEARNING

**Data**
→
**+ Answers**
labels
→
**= Rules**
the model

**We flip the arrow: instead of writing rules, we learn them from examples**

A spam filter trained on 1M emails labeled “spam / not spam” produces a model — a compressed form of the rules. New email arrives → model predicts. You never wrote a single rule.

## WHAT IS MACHINE LEARNING?

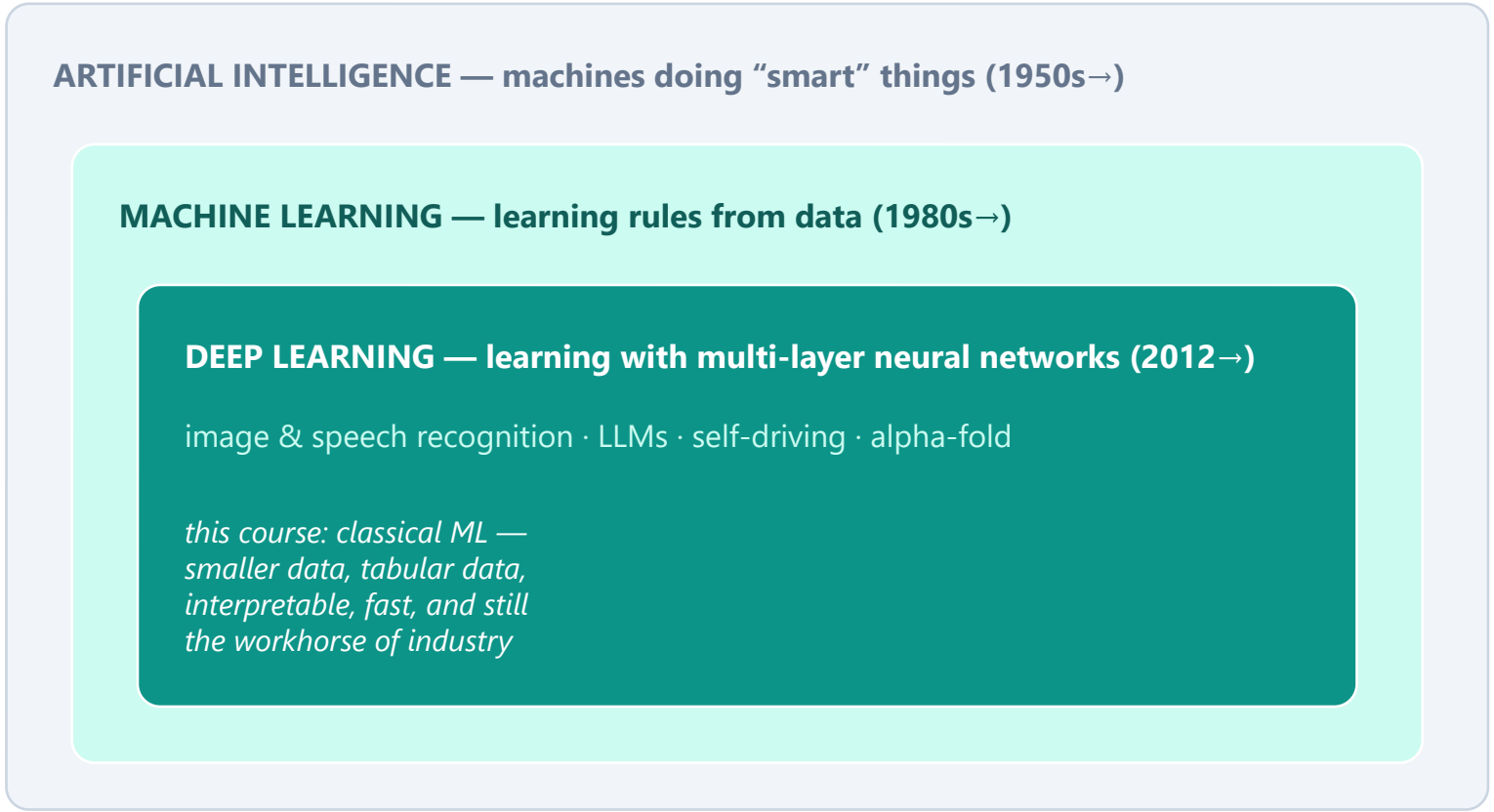
*"A computer program is said to learn from experience E, if its performance at task T,*
*measured by performance P, improves with E."*

— Tom Mitchell, Machine Learning (1997)

```
spam filter:   T = classify emails · E = past labeled emails · P = % correctly classified
```

THE MAP

# AI ⊃ ML ⊃ Deep Learning




- **This course = classical ML**
- scikit-learn's home turf
- Tabular data: rows × columns
- Trains in seconds, not hours
- You can explain why it decided
  - Deep learning → later course (fastai, PyTorch)

TAXONOMY

# Three families of machine learning

## Supervised

Learn from labeled examples: (X → y)

- Will it rain? (yes/no)
- House price? (number)
- Spam or not?

Most business ML lives here.

## Unsupervised

Find structure, no labels given

- Customer segments
- Unusual transactions
- Compress 50 features → 2D

Session 07 covers this.

## Reinforcement

Learn by acting & getting rewards

- Game-playing agents
- Robotics
- Recommendation loops

Out of scope — but you'll meet it again.

SUPERVISED

# Classification vs regression — know your y

### Classification — y is a category

- churn: yes / no
- disease: benign / malignant
- email: spam / ham
- digit: 0–9

Output: a class label, sometimes with probability (79% spam)

### Regression — y is a number

- house price: ฿4.2M
- demand next week: 1,240 units
- patient stay: 5.5 days

Output: a continuous value

(Watch out: “linear regression” is regression; “logistic regression” is classification!)

**Choosing the family = reading your target column y. Everything else in sklearn follows from it.**

WORKFLOW

# The ML workflow you'll repeat all course

**1**
**Frame**
what is y? what's success?

→

**2**
**Collect & EDA**
look at the data first

→

**3**
**Prepare**
clean, encode, scale

→

**4**
**Train**
fit a first quick model

**5**
**Evaluate**
on data it never saw

→

**6**
**Tune**
hyperparameters, features

→

**7**
**Error analysis**
where & why it fails

→

**8**
**Communicate**
insights, not just scores

Sessions map 1:1 onto this loop: 02–03 = prepare · 04 = evaluate · 05–07 = train & tune · 08 = all of it.

TOOLING

# scikit-learn — one library, the whole workflow

| Inside sklearn | what it gives you |
|---|---|
| Datasets | iris, wine, breast cancer, … |
| Preprocessing | impute, encode, scale (S02–03) |
| Models | linear, trees, ensembles, clustering (S05–07) |
| Model selection | train/test split, CV, grid search (S04, 07) |
| Metrics | accuracy, F1, ROC, RMSE, $R^2$ (S04) |

- **Why practitioners love it**
- One consistent API for ~everything
- Excellent docs & tutorials (scikit-learn.org)
- Plays well with pandas
- Not for: deep learning, huge data
  - (this course = classical ML, so perfect fit)

THE ONE API

# Every model: fit() → predict()

**model = Model(hyperparams)**
create — no data yet

→

**model.fit(X_train, y_train)**
learn from examples

→

**y_pred = model.predict(X_test)**
apply to new data

works for ANY sklearn model

```
model = KNeighborsClassifier(n_neighbors=3)
model = DecisionTreeClassifier(max_depth=4)   # swap the model,
model = RandomForestClassifier()              # keep the rest:
model.fit(X_train, y_train)                   # same 3 lines
model.predict(X_test)
```

- **Transformers too**
- scaler.fit(X) → learn the stats
- scaler.transform(X) → apply
  - fit_transform() = both at once
- Consistency is why sklearn won: learn the API once, use 50+ models.

DATASET

# Meet the iris dataset (1936!)

```
from sklearn.datasets import load_iris
X, y = load_iris(return_X_y=True)   # 150 flowers × 4 features
```

| y | sepal L | sepal W | petal L | petal W |
|---|---|---|---|---|
| 0 | 5.1 | 3.5 | 1.4 | 0.2 |
| 0 | 4.9 | 3.0 | 1.4 | 0.2 |
| 1 | 7.0 | 3.2 | 4.7 | 1.4 |
| 2 | 6.3 | 3.3 | 6.0 | 2.5 |

150 flowers · 3 species (0=setosa, 1=versicolor, 2=virginica) · 4 measurements in cm

- **X = features (4 columns)**
- measurements — the inputs
- **y = target (1 column)**
- species 0/1/2 — the answer
- **Goal: predict species from measurements → classification**

MOMENT OF TRUTH

# Your first model — five lines

```
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42)      # ① hold out 30%
model = KNeighborsClassifier(n_neighbors=3)    # ② create
model.fit(X_train, y_train)                    # ③ learn
print(model.score(X_test, y_test))             # ④ judge
```

```
1.0
```

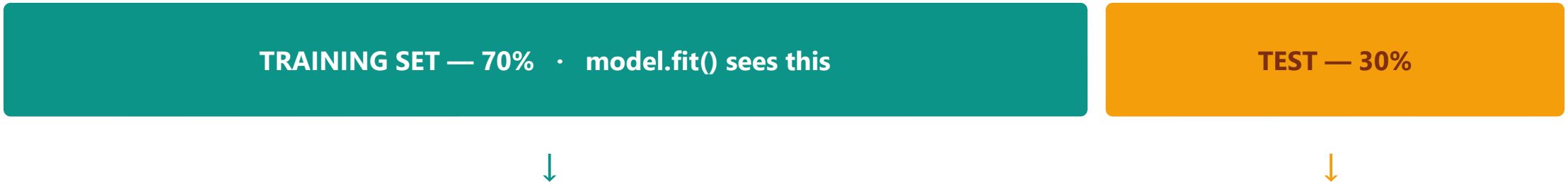
## 100% right

on flowers it never saw

- score() on the TEST set — the model never trained on it
- Iris is gently easy. Real data (Titanic, next session) won't hand you 100%.

KEY HABIT

# Never grade a student on questions it has seen




### fit → learn patterns

The model adjusts itself to minimize mistakes on the training set. Like studying with the answer key.

### score → honest exam

Test questions were never shown. This number estimates real-world performance.

random_state=42 makes the split reproducible — same split on every machine. Full story in Session 04.

INSPECT

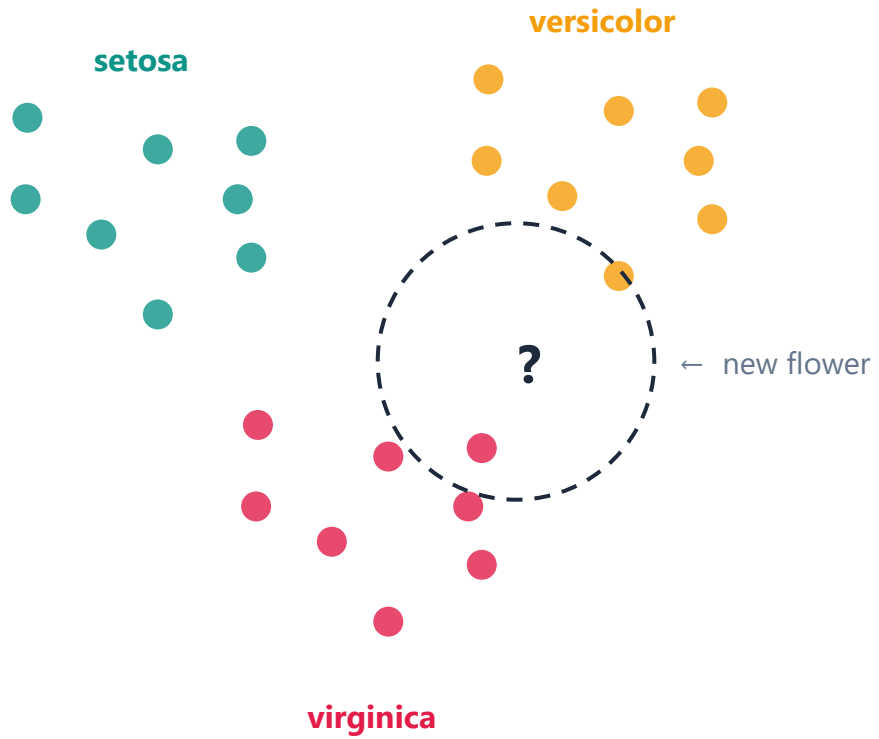
# Look at what it got wrong

```
model50 = KNeighborsClassifier(n_neighbors=50).fit(X_train,
y_train)
y_pred = model50.predict(X_test)
compare = pd.DataFrame({"actual": y_test, "predicted": y_pred})
compare[compare.actual != compare.predicted]      # only the
misses
```

| actual | predicted |
|---|---|
| 2 | 1 |
| 2 | 1 |

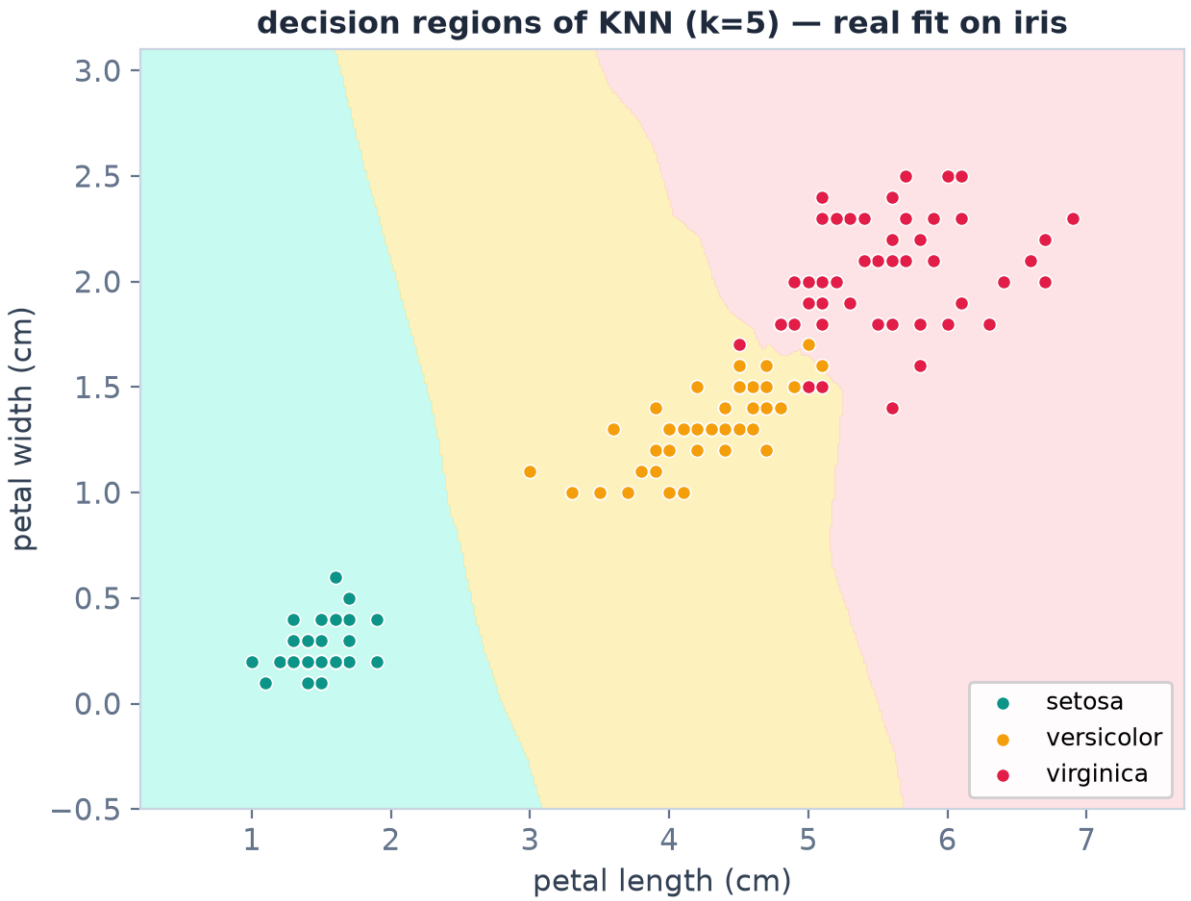
- **k=50 → 2 misses out of 45: virginica (2) predicted as versicolor (1)**
- These two species overlap in measurements — even humans misjudge them
- Error analysis (S07) = reading these misses systematically
- predict() returns labels; predict_proba() returns probabilities

# KNN in 30 seconds — “you are like your neighbors”




- **To classify a new point:**
- ① find its k nearest training flowers
- ② let them vote on the species
- **k=3 here → 2× versicolor wins**
- **k is a hyperparameter — YOUR choice before training**
- small k: flexible, noisy · large k: stable, bland

# What KNN actually learns — decision regions




- **The model IS this map**
- every point in the plane inherits its neighbors' vote
- the wiggly border = KNN's local, memory-based nature
- **k small → wigglier · k large → smoother (S04 measures it)**
- real fit: iris, petal length × width, k=5
- in 30 features it's the same idea — we just can't draw it

EXPERIMENT

# Turn one knob, watch the score move

```
for k in [1, 3, 5, 15, 50]:
    m = KNeighborsClassifier(n_neighbors=k)
    m.fit(X_train, y_train)
    print(f"k={k:2d}  accuracy={m.score(X_test, y_test):.3f}")
```

- **Iris is easy — every sane k is perfect**
- so we crank k until it breaks:
- **k=50 → too blunt**
- votes drown the local pattern (underfitting)
- **On messier data the right k matters a lot — picking it properly = cross-validation, Session 04**

```
k= 1  accuracy=1.000
k= 3  accuracy=1.000
k= 5  accuracy=1.000
k=15  accuracy=1.000
k=50  accuracy=0.956
```

CHEAT SHEET

# Which model when? (simplified sklearn map)

| Situation | Start with | Then try | Session |
|---|---|---|---|
| Few rows, need explainability | Linear / Logistic Regression | Decision Tree | 05 / 06 |
| Tabular, want best accuracy | RandomForest / HistGradientBoosting | XGBoost, LightGBM | 06 |
| Small clean data, smooth boundaries | KNN | SVM (RBF) | 06 |
| No labels — find groups | KMeans | DBSCAN, PCA | 07 |
| Text features | TfifVectorizer + LogisticRegression | naive_bayes | — |

Full interactive map: scikit-learn.org → “choosing the right estimator”.

# Prompting AI for ML code that actually works

**CONTEXT**
pandas df, columns, dtypes

→

**GOAL**
what you want to predict

→

**CONSTRAINT**
sklearn, no leakage, reproducible

→

**VERIFY**
check vs docs / run it

**a prompt that works**

```
I have a pandas DataFrame `df` with columns
tenure(float), contract(object), monthly_charges(float),
churn(0/1 int). Using scikit-learn only:
1) build a ColumnTransformer that imputes+one-hot-encodes
   contract and scales the numeric columns,
2) chain it with LogisticRegression in a Pipeline,
3) evaluate with 5-fold stratified CV — mean ROC-AUC.
Random-state 42. Do NOT touch a holdout test set.
```

- **Why this works:**
- names dtypes → encoder choice is right
- "Pipeline" → leakage-safe by design
- "CV" → honest estimate, no test leakage
- **Then verify: imports real? params real?**

VIBE CODING

# Four ways an AI assistant will lie to you

### Invented API

sklearn.magic.AutoML(...) doesn't exist. LLMs interpolate plausible names. Paste into an editor and it fails — trust only what runs.

### Silent leakage

“I improved your accuracy!” — by fitting the scaler on ALL data or scoring on the training set. Always ask: what did fit() see?

### Deprecated args

train_test_split(train_size=0.7, test_size=0.7)? graphviz syntax changed? APIs move. Check the version in the docs.

### Metric confusion

accuracy on a 99%-negative problem looks like 99%. LLMs rarely warn you. Session 04 gives you the questions to ask.

**Defense: run it, read it line by line, and check anything unfamiliar against scikit-learn.org.**

LAB 01 · DO IT IN GOOGLE COLAB

# โมเดลแรกของคุณ — First model, your hands

**YOUR MISSION (โจทย์ภาษาไทยใน Colab)**

- โหลด iris แล้วแบ่ง train/test 70:30 (random_state=42)
- เทรน KNeighborsClassifier (k=3) และรายงานคะแนนบนชุดทดสอบ
- รันเปลี่ยน k ∈ {1,3,5,15,50} — ผลลัพธ์เปลี่ยนอย่างไร?
- สร้างตารางเทียบ actual vs predicted แล้วหาแถวที่ทายผิด
- โจทย์ชวนคิด: ทำซ้ำทั้งหมดกับ wine dataset (load_wine) โดยไม่ดูเฉลย

## Learning goal

Train, score and inspect your own KNN model on real data — and transfer the pattern to a new dataset.

## Vibe coding corner

ลองถาม AI: “อธิบายว่าทำไม k=50 ให้คะแนนแย่กว่า k=3” แล้วตรวจสอบคำตอบกับสไลด์หน้า 17 — AI อธิบายเหตุผลได้ดี แต่คุณต้องเป็นคนตัดสินว่าเชื่อไหม

## KEY TAKEAWAYS

1. ML learns rules from (data + answers) — for problems you can't hand-code.
2. Supervised = learn X→y; classification (category) vs regression (number).
3. sklearn's one API: create → fit() → predict() — same 3 lines for every model.
4. Always split train/test: score on data the model never saw.
5. AI assistants accelerate you — verify APIs, watch for leakage, explain every line.

## NEXT

Session 02 — Data Understanding & Cleaning — real data is messy